

prompts_v2 — Chart Selection Rules (Governance Format)

General rule → specific rule → exception, one file per chart mechanism. Sourced from `internal/mcpagent/prompts_v2/`. Query numbers reference the merged `scripts/adoption_chart/cto_chart_ideas.html` catalogue (1–30).

§1 — Trend over time

§1.0 General rule

When the question asks whether a metric is changing over time, and the metric is noisy day-to-day, never render the raw series alone. Layer a smoothing signal (rolling average, or a percentile band) on top of the raw line/area so the reader can tell a real trend from daily noise. This general rule is refined by the specific rules below, one per query shape that has actually been asked.

§1.1 Specific rule — "Is LiveReview adoption increasing since my team started using it?" (query #1)

Where the data lives:

- **Table:** `reviews` — one row per review.
- **Date column:** use whichever of `completed_at` / `created_at` is populated (fall back from the first to the second), since not every review has completed.
- **Count:** number of review rows per calendar day.
- **Window:** a trailing stretch long enough for a 7-day average to mean something — 90 days is the usual choice.
- **Zero-fill the calendar.** Days with no reviews must appear as 0, not go missing. Generate the date series and left-join the counts onto it, otherwise a quiet week silently closes up and the trend looks better than it was.
- **The smoothing and the baseline are the query's job, not the chart's.** Compute the 7-day rolling average and the period average as window functions alongside the daily count, so the chart just plots columns that already exist.

Vega-Lite spec (3 layers — area of raw daily reviews, 7-day rolling average line, dashed period-average rule):

```
{
  "width": 900, "height": 420,
  "layer": [
    {"mark": {"type": "area", "opacity": 0.25, "color": "#7c9cff", "interpolate": "monotone"},
      "encoding": {"x": {"field": "day", "type": "temporal"}, "y": {"field": "reviews", "type": "quantitative"}}},
    {"mark": {"type": "line", "color": "#ffb454", "strokeWidth": 2.5, "interpolate": "monotone"},
      "encoding": {"x": {"field": "day", "type": "temporal"}, "y": {"field": "rolling_avg_7d", "type": "quantitative"}}},
    {"mark": {"type": "rule", "color": "#ff5c7c", "strokeDash": [6, 4], "strokeWidth": 1.5},
      "data": {"values": [{"period_avg": "<period_avg>"}]},
      "encoding": {"y": {"field": "period_avg", "type": "quantitative"}}}
  ],
  "resolve": {"scale": {"y": "shared"}}
}
```

§1.2 Specific rule — "What happened to a repository's velocity?" (query #10)

- Refines §1.1: same rolling-average mechanism, but on **daily LOC for one named repository** (not org-wide review count), plus a highlighted recent-interval rectangle layer §1.1 does not have.

Where the data lives:

- **Tables:** `loc_usage_ledger` joined to `reviews` — the ledger carries the lines-of-code figure, `reviews` carries the repository name you filter on.
- **Measure:** billable LOC summed per day, not a review count. "Velocity" is about how much code moved, and one large review is not the same event as one trivial one.
- **Ledger rows only count when they are settled** — filter to accounted status, or you will mix provisional numbers into the history.
- **Window and zero-fill:** same as §1.1 — daily granularity, trailing ~90 days, gaps filled with 0.
- **Rolling average:** 7-day window function in the query.

- The highlighted interval (the last two weeks) is a chart-side band you supply as two dates; it does not come from this query.

Vega-Lite spec (3 layers — highlight rect, thin raw line, heavier rolling average line):

```
{
  "width": 800, "height": 340,
  "layer": [
    {"data": {"values": [{"start": "<highlight_start>", "end": "<highlight_end>"}]},
    {"mark": {"type": "rect", "color": "#7c9cff", "opacity": 0.12},
     "encoding": {"x": {"field": "start", "type": "temporal"}, "x2": {"field": "end"}}},
    {"mark": {"type": "line", "color": "#3a4358", "strokeWidth": 1},
     "encoding": {"x": {"field": "day", "type": "temporal"}, "y": {"field": "loc", "type": "quantitative"}}},
    {"mark": {"type": "line", "color": "#ffb454", "strokeWidth": 2.5},
     "encoding": {"x": {"field": "day", "type": "temporal"}, "y": {"field": "rolling_avg", "type": "quantitative"}}}
  ],
  "resolve": {"scale": {"y": "shared"}}
}
```

§1.3 Specific rule — "How much engineering work is being covered by LR?" (query #23)

- Variant of §1.0's general rule: two independent daily metrics on **independent y-scales** (not one metric + its own smoothed self) — LOC and review count plotted together to show whether "more reviews" means "more code inspected" or just "more, smaller reviews."

Where the data lives:

- **Two separate daily aggregates**, joined onto one shared date series: review count from `reviews`, and summed billable LOC from `loc_usage_ledger` (settled rows only).
- Aggregate each one **before** joining them together. Counting rows after a join between the two would multiply reviews by their ledger entries and inflate both numbers.
- Same daily granularity, trailing window and zero-fill as §1.1.
- No rolling average here — the second line *is* the comparison, so smoothing is not what makes the chart readable.

Vega-Lite spec (2 line layers, independent y-scales, one axis left/one right):

```
{
  "width": 800, "height": 340,
  "layer": [
    {"mark": {"type": "line", "color": "#ffb454", "strokeWidth": 2},
     "encoding": {"x": {"field": "day", "type": "temporal"}, "y": {"field": "loc", "type": "quantitative", "axis": "left"}},
    {"mark": {"type": "line", "color": "#7c9cff", "strokeWidth": 2},
     "encoding": {"x": {"field": "day", "type": "temporal"}, "y": {"field": "reviews", "type": "quantitative", "axis": "right"}}}
  ],
  "resolve": {"scale": {"y": "independent"}}
}
```

§1.4 Exception — "Are reviews getting faster?" (band, not average) (query #22)

- **This is an exception to §1.0, not another instance of it.** §1.0 smooths a single noisy line. This question is not about the center of the metric at all — it's about whether the *tail* (p90) is getting worse even if the median (p50) looks fine. A rolling average would hide exactly the signal this question needs, so the correct layer is an `errorband` (p10–p90) plus a median line, not a rolling-average line.

Where the data lives:

- **Table:** `reviews`, restricted to reviews that actually finished — both a completed status and a non-null completion timestamp, since an unfinished review has no duration.
- **Measure:** duration is the gap between creation and completion. Derive it per review, then convert to a unit a person reads easily (minutes or hours, not raw seconds).
- **Three numbers per bucket, not one:** the median plus a low and high percentile. Postgres computes these with an ordered-set aggregate over the per-review duration.
- **Bucket weekly, not daily.** Percentiles need enough reviews inside a bucket to be stable; a daily p90 over three reviews is noise wearing a statistic's clothing.

Vega-Lite spec (errorband + median line):

```
{
  "width": 700, "height": 340,
  "layer": [
    {"mark": {"type": "errorband", "color": "#7c9cff", "opacity": 0.25},
     "encoding": {"x": {"field": "week", "type": "temporal"}, "y": {"field": "p10", "type": "quantitative"}, "y2": {"field": "p90", "type": "quantitative"}},
    {"mark": {"type": "line", "color": "#ffb454", "strokeWidth": 2.5, "point": true},
     "encoding": {"x": {"field": "week", "type": "temporal"}, "y": {"field": "p50", "type": "quantitative"}}}
  ]
}
```

§2 — Calendar / activity-rhythm heatmap

§2.0 General rule

When the question asks about a pattern across calendar days — habit, rhythm, consistency, gaps, bursts — not "who did the most," render a GitHub-style calendar grid, not a time-series line. Data must be daily-granularity and zero-filled for missing days (a missing row renders as nothing, not a colored "zero" cell — see §2.1's zero-fill CTE). This is the one pattern

`internal/mcpagent/prompts/analytics_plan.md` already has an explicit exception for (its rhythm/habit/consistency rule) — see `PROMPT_LOGIC.md` §2.3.

§2.1 Specific rule — "Are engineers actually incorporating reviews into their daily workflow?" (query #2)

- Axes: x = week (ordinal, not temporal — see note in spec), y = day-of-week (Mon/Wed/Fri labeled only), color = review count, GitHub-green threshold scale, no legend.

Where the data lives:

- **Table:** `reviews`, counted per calendar day (or summed LOC via `loc_usage_ledger` if the question is about volume rather than frequency).
- **Zero-fill is not optional here — it is the whole chart.** This visualization exists to show *gaps*: the weekends, the abandoned fortnight, the day someone stopped. A missing row draws nothing at all, so an empty day would render as blank paper instead of a dark cell, and the one signal the reader came for disappears. Generate the full date series and left-join onto it.
- **Window:** long enough to see a rhythm — 90 days or more. A two-week window cannot show a habit.
- **Grouping:** by day, never by author. "Is this a habit" is a question about the calendar; who did it is a different question (see §4).

Vega-Lite spec:

```
{
  "width": {"step": "<cell_step>"}, "height": {"step": "<cell_step>"},
  "mark": {"type": "rect", "cornerRadius": 2},
  "encoding": {
    "x": {"field": "day", "type": "ordinal", "timeUnit": "yearweek",
      "scale": {"paddingInner": 0.15},
      "axis": {"format": "%b", "labelExpr": "date(datum.value) <= 7 ? timeFormat(datum.value, '%b') : ''", "tickExpr": "date(datum.value) <= 7 ? timeFormat(datum.value, '%b') : ''"}},
    "y": {"field": "day", "type": "ordinal", "timeUnit": "day",
      "sort": ["Sun", "Mon", "Tue", "Wed", "Thu", "Fri", "Sat"],
      "scale": {"paddingInner": 0.15},
      "axis": {"values": ["Mon", "Wed", "Fri"]}},
    "color": {"field": "value", "type": "quantitative",
      "scale": {"type": "threshold", "domain": [1, 3, 6, 10], "range": "<github_greens>"},
      "legend": null}
  }
}
```

Why ordinal, not temporal (a rule inside this rule): a temporal x-scale here puts week columns on a continuous time scale whose bandwidth math for a `rect` mark collapses/overlaps adjacent weeks. Ordinal is mandatory for both x and y in this pattern.

§2.2 Specific rule — "What does engineering activity look like across repositories and days?"

(query #9)

- Refines §2.0: same `rect` + `color` mechanism, but the y-axis is **repository** (sorted by total LOC) instead of day-of-week, and x stays a plain temporal day axis rather than §2.1's ordinal year-week banding — because this chart is not trying to show a Mon–Sun weekly rhythm, it's comparing repos against each other over a continuous window.

Where the data lives:

- Tables:** `loc_usage_ledger` joined to `reviews` for the repository name; settled ledger rows only.
- Grouping:** two keys — repository and day — giving one row per cell of the grid.
- Sort the repositories by their total**, so the busiest sit together rather than scattering alphabetically. Let the chart do this from the data rather than hardcoding a repository list, which goes stale the moment a new repo appears.
- Zero-fill matters less here than in §2.1: with dozens of repositories the grid is mostly empty by nature, and filling every repo × day pair would balloon the result for little gain.

Vega-Lite spec:

```
{
  "width": {"step": 14}, "height": {"step": 26},
  "mark": {"type": "rect"},
  "encoding": {
    "x": {"field": "day", "type": "temporal", "axis": {"format": "%b %d", "labelAngle": -40}},
    "y": {"field": "repository", "type": "nominal", "sort": "<repos-by-loc-desc>"},
    "color": {"field": "loc", "type": "quantitative", "scale": {"scheme": "blues"}}
  }
}
```

§3 — Distribution across a population

§3.0 General rule

When the question asks how spread out a metric is across many rows — not the total, not the ranking, but the shape of the spread — bucket the metric and render a histogram (`bar` over SQL-computed bins), or keep every point visible with a jittered strip/beeswarm plot if the population is small enough to show individually (roughly under ~30 points).

§3.1 Specific rule — "How broadly has the organization adopted LiveReview?" (query #3)

- Bucketing happens in Python (`band_for()` in `generate_breadth.py` , shared with §4/§1's leaderboard/growth charts), not in SQL — bands are 1-4 (light) / 5-19 (regular) / 20+ (heavy) .

Where the data lives:

- Table:** `reviews` , grouped by author to get one row per engineer with their review count.
- Skip rows with no author.** Automated and system-triggered reviews have no person attached, and counting them as an anonymous "engineer" invents a teammate who does not exist.
- Window:** a trailing 90 days. Adoption questions are about the current state of the team, and all-time totals let someone who left last year keep looking active.
- Bucketing into light / regular / heavy can happen in the query or after it** — either is fine, but the band thresholds must be the same ones §4 uses, or "heavy user" quietly means two different things on two charts in the same conversation.

Vega-Lite spec:

```
{
  "width": 700, "height": 340,
  "mark": {"type": "bar", "cornerRadiusTopLeft": 4, "cornerRadiusTopRight": 4},
  "encoding": {
    "x": {"field": "band", "type": "nominal", "sort": "<band_order>", "axis": {"labelAngle": 0}},
    "y": {"field": "engineers", "type": "quantitative"},
    "color": {"field": "band", "type": "nominal", "sort": "<band_order>",
      "scale": {"domain": "<band_order>", "range": "<color_range>", "legend": null}}
  }
}
```

§3.2 Specific rule — "Are reviews becoming more iterative?" (query #24)

- Binning happens in SQL this time (nested `GROUP BY`), not in application code — the distinction from §3.1 is where the bucketing lives, not the chart mechanism, which is identical (`bar`, ordinal $x = \text{bucket}$, $y = \text{count}$).

Where the data lives:

- **Table:** `reviews`, keyed on the commit identifier.
- **This is a count of counts — two aggregation passes.** First: how many reviews each commit received. Then: how many commits received each of those numbers. Skipping the second pass gives you a list of commits, which is data, not a distribution.
- **Exclude rows with no commit recorded**, otherwise they collapse into one fake mega-commit.
- The x-axis values here are small integers (1, 2, 3 reviews), so they are already their own buckets — no band thresholds needed.

Vega-Lite spec:

```
{
  "width": 600, "height": 340,
  "mark": {"type": "bar", "color": "#7c9cff"},
  "encoding": {
    "x": {"field": "reviews_per_commit", "type": "ordinal"},
    "y": {"field": "commits", "type": "quantitative"}
  }
}
```

§3.3 Exception — "Which engineers are carrying the repository?" (keep every point visible) (query #12)

- **Exception to §3.0's binning default:** when the population is small (one repository's contributor list, not the whole org) and outliers themselves are the point of the question, do not bin at all — render every engineer as a jittered point (`circle` + `yoffset` on a `random()` calculate transform) so no individual gets flattened into a bucket average.

Where the data lives:

- **Tables:** `loc_usage_ledger` joined to `reviews`, filtered to one repository and to authored (non-null) reviews.
- **Two measures per engineer:** LOC reviewed and review count. LOC drives the position on the axis; the count drives the dot size, so a person who did a lot of small reviews reads differently from one who did a few enormous ones.
- **No bucketing.** One row per engineer is the point — see the exception note above.
- Sort by the larger measure so the heaviest contributors are adjacent.

Vega-Lite spec:

```
{
  "width": 600, "height": "<32 * n_engineers, min 200>",
  "transform": [{"calculate": "random()", "as": "jitter"}],
  "mark": {"type": "circle", "opacity": 0.85},
  "encoding": {
    "x": {"field": "loc", "type": "quantitative"},
    "y": {"field": "engineer", "type": "nominal", "sort": "-x"},
    "y0offset": {"field": "jitter", "type": "quantitative"},
    "size": {"field": "reviews", "type": "quantitative", "scale": {"range": [60, 900]}},
    "color": {"field": "loc", "type": "quantitative", "scale": {"scheme": "blues"}, "legend": null}
  }
}
```

§4 — Ranked leaderboard

§4.0 General rule

When the question ranks named entities against each other (who did the most/least), render **ONE sorted horizontal bar** — never split "most" and "least" into two separate charts (they are the two ends of the same ranking, not two questions). Add a dashed target-rule layer when a threshold is meaningful, and color by tier when a banded read (light / regular / heavy) adds more than a plain sort does.

§4.1 Specific rule — "Who has adopted LiveReview the most and least?" (query #4)

- Metric is configurable (`--metric reviews|loc`); default target is 5 reviews / 200 LOC over the trailing window. Uses the same `BANDS / band_for()` tiering as §3.1 so a "tier" means the same thing across both charts.

Where the data lives:

- **Ranking by review count:** `reviews` grouped by author. **Ranking by LOC:** `loc_usage_ledger` joined to `reviews` for the author name, settled rows only. Either is defensible — count answers "who is using it", LOC answers "who is putting real work through it".
- **Drop rows with no author** for the same reason as §3.1.
- **Trailing 90 days**, not all time.
- **The "who hasn't" half of the question is the hard half.** Grouping the reviews table alone can only ever list people who *did* review — anyone at zero is absent from the result and therefore absent from the chart, which is precisely the person the question was asked about. To answer it honestly you need the org's member roster as the left side of the join, with review counts filled in as zero where there is no match. If no roster is reachable, say plainly in the description that the chart shows only engineers with at least one review, rather than letting silence imply everyone is on it.
- **The target line is a constant you supply**, not a queried value.

Vega-Lite spec (2 layers — sorted bar colored by tier, + dashed target rule):

```
{
  "width": 700, "height": "<max(200, 28 * n_engineers)>",
  "layer": [
    {
      "mark": {
        "type": "bar",
        "cornerRadiusTopRight": 3,
        "cornerRadiusBottomRight": 3,
        "encoding": {
          "y": {
            "field": "engineer",
            "type": "nominal",
            "sort": "-x"
          },
          "x": {
            "field": "value",
            "type": "quantitative"
          },
          "color": {
            "field": "band",
            "type": "nominal",
            "scale": {
              "domain": "<band_order>",
              "range": "<color_range>",
              "legend": null
            }
          }
        }
      },
      "data": {
        "values": [{"target": "<target>"}]
      },
      "mark": {
        "type": "rule",
        "color": "#ff5c7c",
        "strokeDash": [6, 4],
        "strokeWidth": 1.5,
        "encoding": {
          "x": {
            "field": "target",
            "type": "quantitative"
          }
        }
      }
    ],
    "resolve": {
      "scale": {
        "y": "shared"
      }
    }
  ]
}
```

§5 — Concentration / Pareto

§5.0 General rule

When the question asks whether a total is dominated by a few contributors ("who accounts for most of X", "is this concentrated or broad"), render a sorted bar plus a second **line** layer of cumulative percent, on an independent right-hand y-scale. This is the one pattern that answers "do our top 3 repos/engineers account for 80% of everything" directly — a plain sorted bar (§4's leaderboard) shows rank but not concentration.

§5.1 Specific rule — "Where is organizational velocity concentrated?" (by repository) (query #7)

Where the data lives:

- **Tables:** `loc_usage_ledger` joined to `reviews` for the repository name; settled rows only; trailing 90 days.
- **Measure:** summed LOC per repository, sorted descending. The sort is not cosmetic — a Pareto curve is meaningless unless the bars are in descending order, because the cumulative line assumes it.
- **The cumulative percentage is a query column, not a chart feature.** Compute a running total over the sorted rows and divide by the grand total. Two window functions over the same ordering.
- Expect a long tail of near-zero repositories. That tail is part of the message ("most of these are dormant"), so do not filter it away — but do quote the top-N share in the description, because that is the number someone will repeat in a meeting.

Vega-Lite spec (bar + cumulative-% line, independent y-scales):

```
{
  "width": 600, "height": 360,
  "layer": [
    {"mark": {"type": "bar", "color": "#7c9cff"},
      "encoding": {"x": {"field": "repository", "type": "nominal", "sort": "-y"}, "y": {"field": "loc", "type": "quantitative", "sort": "-y"}, "y2": {"field": "cum_pct", "type": "quantitative", "axis": {"orient": "right"}}}},
    {"mark": {"type": "line", "point": true, "color": "#ff5c7c", "strokeWidth": 2},
      "encoding": {"x": {"field": "repository", "type": "nominal", "sort": "-y"},
        "y": {"field": "cum_pct", "type": "quantitative", "axis": {"orient": "right"}}}},
  ],
  "resolve": {"scale": {"y": "independent"}}
}
```

§5.2 Specific rule — "How much of the organization's activity is covered by the top users?" (by engineer) (query #29)

- Identical mechanism to §5.1 — only the grouping column changes (`author_username` instead of `repository`). This is the confirmed near-duplicate flagged in the chart-idea grouping review: same shape, same two-layer spec, only the entity being ranked differs.

Where the data lives: identical in shape to §5.1 — same descending sort, same running-total-over-grand-total for the cumulative line — but grouped on the author column of `reviews` instead of the repository, and counting reviews rather than summing LOC.

Vega-Lite spec: identical structure to §5.1, `x.field = engineer`, `y.field` (bar layer) = `reviews`.

Consolidation note: §5.1 and §5.2 are the same rule with a parameterized entity column (`repository` vs `engineer`). A future revision of this framework should probably merge them into one rule with an `entity` parameter rather than maintaining two near-identical specific rules.

§6 — Period-over-period comparison

§6.0 General rule

When the question asks whether something is "gaining or losing," "went up or down," or "changed between two points in time" — collapse the window into exactly two buckets (Previous/Current, W1/W2) in SQL, never leave it as a multi-bucket trend. A monthly/weekly trend answers a different question ("how has this moved continuously") than a two-period comparison ("where do we stand now vs. before"). This is the general rule `analytics_plan.md` is missing entirely today — see `PROMPT_LOGIC.md` §2's gap table.

§6.1 Specific rule — "Which repositories are gaining or losing engineering velocity?" (query #6)

- Shape: slope graph — one line per repository between exactly two x points (`Previous` , `Current`), colored by direction.

Where the data lives:

- Tables:** `loc_usage_ledger` joined to `reviews` ; settled rows only.
- The key move is labelling each row Previous or Current** with a conditional expression on its date, then grouping by repository *and* that label. Take a window — 90 days is comfortable — and split it down the middle, so both halves are the same length and the comparison is fair.
- Exactly two buckets.** Not six months of monthly points: the question is "where do we stand now versus before", and a multi-bucket trend forces the reader to eyeball a direction that the chart should be stating outright.
- Derive the gain/loss label per repository** by comparing its two values, and encode that as the line colour. That label is what turns a tangle of lines into an answer.

Vega-Lite spec:

```
{
  "width": 500, "height": 380,
  "mark": {"type": "line", "point": true, "strokeWidth": 2.5},
  "encoding": {
    "x": {"field": "period", "type": "nominal", "sort": ["Previous", "Current"]},
    "y": {"field": "loc", "type": "quantitative"},
    "color": {"field": "trend", "type": "nominal",
      "scale": {"domain": ["gain", "flat", "loss"], "range": ["#39d353", "#8b949e", "#ff5c7c"]}},
    "detail": {"field": "repository", "type": "nominal"}
  }
}
```

§6.2 Specific rule — "What changed between week 1 and week 2?" (query #30)

- Refines §6.0 for **multiple metrics at once** (reviews, LOC, active engineers, repos, ...), not one metric for many entities like §6.1 — a `rect + color` heatmap (metric × period, color = delta) with a `text` layer overlaying the actual numbers, instead of a slope line per entity.

Where the data lives:

- Several small queries, not one big one.** Each row of the matrix is a different metric from a different place — review counts from `reviews`, LOC from `loc_usage_ledger`, active engineers as a distinct author count, repositories touched, trigger-type share. There is no single join that produces all of them sensibly.
- Run each metric for both halves of the window**, then assemble the results into one small table of metric / period / value rows.
- The delta is a third "period" column**, computed once you have the two values — that is what the colour scale reads.
- Keep the metric list short and stable. This chart is a verdict, and a verdict with fifteen rows is a spreadsheet.

Vega-Lite spec (2 layers — delta-colored rect, text overlay):

```
{
  "width": 300, "height": 220,
  "layer": [
    {"mark": {"type": "rect"},
      "encoding": {
        "x": {"field": "period", "type": "nominal", "sort": ["W1", "W2", "Delta"]},
        "y": {"field": "metric", "type": "nominal", "sort": "<metric_order>"},
        "color": {"condition": {"test": "datum.is_delta", "field": "value", "type": "quantitative",
          "scale": {"domainMid": 0, "range": ["#ff5c7c", "#232a3d", "#39d353"]}},
          "value": "#161b22"}
      }},
    {"mark": {"type": "text", "color": "#e6ebf5", "fontSize": 12},
      "encoding": {
        "x": {"field": "period", "type": "nominal", "sort": ["W1", "W2", "Delta"]},
        "y": {"field": "metric", "type": "nominal", "sort": "<metric_order>"},
        "text": {"field": "value", "type": "quantitative", "format": ".1f"}
      }
    ]
}
```

§6.3 Exception — "Why did this repository's velocity change?" (decomposition, not just the delta) (query #11)

- Exception to §6.0/§6.1:** "why" is not answered by showing that velocity changed (§6.1 already does that) — it needs the change decomposed per contributor, so a sorted diverging bar (one bar per engineer, colored up/down) is used instead of a slope line or a metric×period matrix.

Where the data lives:

- Same two-period split as §6.1 and the same tables, but grouped by **author** and filtered to **one repository** — the question is about what happened inside a single repo.
- Then subtract:** one row per engineer carrying the change between their two periods, plus the direction of that change for the colour. The chart plots the delta, not the two raw values, so the arithmetic belongs in the query.
- Sort by the delta so the biggest movers sit at the ends, which is where the eye goes first.

Vega-Lite spec:

```
{
  "width": 600, "height": "<max(200, 30 * n_engineers)>",
  "mark": {"type": "bar"},
  "encoding": {
    "y": {"field": "engineer", "type": "nominal", "sort": "x"},
    "x": {"field": "delta", "type": "quantitative"},
    "color": {"field": "direction", "type": "nominal", "legend": null,
      "scale": {"domain": ["up", "down"], "range": ["#39d353", "#ff5c7c"]}}
  }
}
```

§7 — Relationship between two numeric measures

§7.0 General rule

When the question compares two (or three) independent numeric measures against each other per entity — not one measure ranked, but "how do X and Y relate" — render a scatter/bubble chart: **circle** mark, **x/y** = the two measures, **size** for a third measure if one exists, **color** for the entity or a fourth dimension. A single-metric bar chart cannot show this relationship even if it uses the "right" metric, because it only has one axis.

§7.1 Specific rule — "Which repositories are unusually active or inactive?" (query #8)

- Measures: LOC reviewed (x) vs. review count (y) — separates high-volume/high-frequency repos from large-diff/low-frequency ones. **size** = active engineers.

Where the data lives:

- Tables:** `reviews` as the base, with `loc_usage_ledger` **left** joined onto it. Left, not inner: a repository with reviews but no settled ledger rows still belongs on the chart at zero LOC, and an inner join would silently delete exactly the "unusually inactive" repos the question asks about.
- Three measures per repository**, all from the same grouped query: review count, distinct author count, and summed LOC (treat a null sum as zero).
- Distinct-counting the authors is what makes the bubble size meaningful — it separates "one person's private repo" from "the whole team's".
- Trailing 90 days.

Vega-Lite spec:

```
{
  "width": 600, "height": 380,
  "mark": {"type": "circle", "opacity": 0.85},
  "encoding": {
    "x": {"field": "loc", "type": "quantitative"},
    "y": {"field": "reviews", "type": "quantitative"},
    "size": {"field": "engineers", "type": "quantitative", "scale": {"range": [80, 1200]}},
    "color": {"field": "repository", "type": "nominal", "legend": null}
  }
}
```

§7.2 Specific rule — "Which engineers are getting the most value from LR?" (query #25)

- Measures: reviews (x) vs. up-voted feedback as a proxy for useful findings (y). **size** = LOC, **color** = acceptance %. Separates "heavy users" from "productive users" — the two are not the same thing.

Where the data lives:

- Tables:** `reviews` grouped by author, left joined to both `loc_usage_ledger` (settled rows) and `review_feedback`.
- Ignore retracted feedback** — someone who took their vote back did not vote.
- Count up-votes and down-votes as separate filtered aggregates** in one pass, rather than running two queries and stitching them together.
- Watch the double-join.** Joining two one-to-many tables onto `reviews` in the same query multiplies rows: each review appears once per ledger row *per* feedback row. Count reviews distinctly, and know that the LOC sum inflates the same way. If in doubt, aggregate each

measure on its own and join those results together.

- Up-votes here are a **proxy** for "useful findings", not a measurement of it. Say so in the description rather than presenting it as fact.

Vega-Lite spec:

```
{
  "width": 600, "height": 380,
  "mark": {"type": "circle", "opacity": 0.85},
  "encoding": {
    "x": {"field": "reviews", "type": "quantitative"},
    "y": {"field": "useful_findings", "type": "quantitative"},
    "size": {"field": "loc", "type": "quantitative", "scale": {"range": [60, 900]}},
    "color": {"field": "acceptance", "type": "quantitative", "scale": {"scheme": "greens"}}
  }
}
```

§7.3 Specific rule — "Which repositories have the highest review coverage?" (query #27)

- Measures: reviews/PRs ratio (x, "coverage") vs. LOC reviewed (y). Same mechanism as §7.1 — near-duplicate flagged in the chart-idea grouping review, both are "LOC/volume vs. a second repo-level measure, sized by engineers." The distinguishing value here is the `coverage` ratio itself, joined against `pull_requests`, which §7.1 does not compute.

Where the data lives:

- **Start from `repositories`, not from `reviews`**. Coverage is a ratio against everything that *could* have been reviewed, so the denominator has to come from the repository and pull-request side of the schema.
- **Tables:** `repositories` left joined to `pull_requests` (the denominator), to `reviews` (the numerator), and through those to `loc_usage_ledger`.
- **Note the join key mismatch:** `reviews` reference a repository by **name**, while pull requests reference it by **id**. Join each on its own key rather than assuming one identifier works for both.
- **Count everything distinctly.** Three left joins fan out badly, and a plain count here will report numbers several times too large.
- **Apply the date filters inside the joins, not in a trailing WHERE.** Filtering a left-joined table afterwards quietly turns it into an inner join and drops the zero rows you went to the trouble of preserving.
- The coverage ratio itself (reviews ÷ PRs) is a derived column — compute it once and encode it, rather than making the chart divide.

Vega-Lite spec:

```
{
  "width": 600, "height": 380,
  "mark": {"type": "circle", "opacity": 0.85},
  "encoding": {
    "x": {"field": "coverage", "type": "quantitative"},
    "y": {"field": "loc", "type": "quantitative"},
    "size": {"field": "engineers", "type": "quantitative", "scale": {"range": [60, 900]}},
    "color": {"field": "coverage", "type": "quantitative", "scale": {"scheme": "blues"}}
  }
}
```

§7.4 Exception — "What is the blast radius of issues being caught?" (not built) (query #19)

- Needs `ai_comments.file_path` -level joins that haven't been validated against real data; file-level blast radius is possible, dependency-level would need metadata that doesn't exist today.
- Would follow §7.0's general mechanism (bubble: findings per review × files affected, size = severity/LOC) once the data is confirmed reliable — documented as a pending instance of this rule, not a new one.

§8 — Stacked share-of-total over time

§8.0 General rule

When the question asks how the composition of a total is shifting over time (which category's share is growing/shrinking), render a 100%-stacked mark (`"stack": "normalize"`) with `color = category`, not a plain count per category. A raw stacked count chart

conflates "this category grew" with "overall volume grew" — normalizing to 100% isolates the mix shift, which is what these questions are actually asking about.

§8.1 Specific rule — "Where are reviews happening?" (trigger-source mix, discrete) (query #14)

- Mark: `bar`, weekly buckets — more useful than a single pie chart because periods can be compared side by side.

Where the data lives:

- **Table:** `reviews`, grouped by two keys: the week bucket and the trigger-type column that records how the review was started.
- **Return raw counts, not percentages.** The chart normalises to 100% itself; if you also divide in the query you will normalise twice and get a flat line at 100%.
- **Weekly buckets.** Daily is too jittery to read a mix shift from, and monthly hides the transition you are trying to show.
- Trailing 90 days or so.

Vega-Lite spec:

```
{
  "width": 700, "height": 340,
  "mark": {"type": "bar"},
  "encoding": {
    "x": {"field": "week", "type": "temporal"},
    "y": {"field": "n", "type": "quantitative", "stack": "normalize", "axis": {"format": "%"}},
    "color": {"field": "trigger_type", "type": "nominal"}
  }
}
```

§8.2 Specific rule — "Are we moving review earlier in the development lifecycle?" (trigger-source mix, continuous) (query #15)

- Refines §8.1: **identical SQL**, only the mark changes — `area ( interpolate: monotone )` instead of `bar`, because this question is about a continuous transition/trend in the mix, not a period-by-period comparison. Same `"stack": "normalize"` mechanism.

Where the data lives: identical to §8.1 — same table, same two grouping keys, same raw counts. Only the mark changes.

Vega-Lite spec:

```
{
  "width": 700, "height": 340,
  "mark": {"type": "area", "interpolate": "monotone"},
  "encoding": {
    "x": {"field": "week", "type": "temporal"},
    "y": {"field": "n", "type": "quantitative", "stack": "normalize", "axis": {"format": "%"}},
    "color": {"field": "trigger_type", "type": "nominal"}
  }
}
```

Consolidation note: §8.1 and §8.2 share one SQL query and differ only in `mark.type ( bar vs area )` — a strong candidate for a single rule with a "discrete comparison vs. continuous trend" mark parameter, the same pattern flagged for §5.1/§5.2.

§9 — Two categorical dimensions crossed

§9.0 General rule

When the question crosses two categorical dimensions (not a category over continuous time — that's §2/§8) and asks where a count/rate is concentrated in the resulting grid, render a `rect + color` heatmap, one cell per combination. The working mechanism for this is already proven by §2.2's repository × day heatmap (`repo_day_heatmap.py`) — day there is treated as a plain categorical/temporal axis, not the Mon–Sun rhythm axis §2.1 uses, so the same `rect + color` recipe generalizes directly to other two-category crossings.

§9.1 Specific rule — "Are serious issues being caught before PR/MR?" (severity × trigger) (query

#16)

- Would be: rows = trigger source (pre-commit / PR-MR / MCP / API), columns = severity (critical/high/medium/low), color = finding count/rate.
- **Exception, not a gap in the rule:** `ai_comments.content` stores the comment payload as JSON, not a normalized issue-severity column — this chart is blocked on schema/extraction work, not on chart design. Do not attempt to fake this from a text search over `ai_comments.content`; wait until severity is reliably extractable.

§9.2 Specific rule — "Where are the issues concentrated?" (repository × file) (query #18)

- Would be: rows = file/path, columns = repository, color = finding count, `text` /size for severity, top files per repo only.
- `ai_comments.file_path` does exist and makes file-level concentration directly possible in principle — this one is closer to buildable than §9.1, but hasn't been validated against real dev-DB volume yet (most `ai_comments` rows in the dev DB are currently sparse/empty — see earlier data-availability triage for this catalogue).

Both rules in this section are documented but unbuilt on purpose — per the standing rule for this whole chart catalogue: never fabricate a demo for data that doesn't exist yet in the dev DB. §9.0's mechanism is proven (via §2.2); what's missing is the underlying data, not the chart recipe.

§10 — One-off shapes

§10.0 General rule

These queries don't share a chart mechanism with each other or with §1–§9 — each gets its own specific rule directly, with no general rule above it beyond "match the named technique to the question, don't default to `bar`". Grouping them here is deliberate: it stops future additions from being forced into an ill-fitting §1–§9 bucket just to avoid an "uncategorized" pile — a one-off is a legitimate outcome, not a filing failure.

§10.1 Specific rule — "Is adoption becoming broader over time?" (query #5)

- Shape: stacked `area` (true stack, not normalized like §8) of active engineer counts, colored by adoption tier (same bands as §3.1/§4.1). Closest relative is §8, but the question is "how many people at each depth" (absolute headcount by tier), not "what fraction of total volume" — so it stays unnormalized.

Where the data lives:

- **Table:** `reviews`, grouped by **week and author together** — you need each person's activity level within each week before you can say which tier they were in that week.
- **Then bucket, then count people.** Two steps: assign each engineer-week to a tier using the same thresholds as §3.1 and §4.1, then count how many engineers fall in each tier per week. The chart plots people, not reviews.
- Skipping the per-author grouping and going straight to distinct-user counts per week is the failure this rule exists to prevent: it produces a headcount line that cannot show depth at all.
- Trailing ~180 days, since this is a question about a slow-moving shift.

Vega-Lite spec:

```
{
  "width": 800, "height": 380,
  "mark": {"type": "area", "interpolate": "monotone", "line": {"strokeWidth": 1.5}},
  "encoding": {
    "x": {"field": "week", "type": "temporal"},
    "y": {"field": "engineers", "type": "quantitative", "stack": true},
    "color": {"field": "band", "type": "nominal", "sort": "<band_order>",
      "scale": {"domain": "<band_order>", "range": "<color_range>"}},
    "order": {"field": "band", "sort": "ascending"}
  }
}
```

§10.2 Specific rule — "What does each engineer actually spend their review activity on?" (query #13)

- Shape: stacked `bar`, `x` = engineer, `color` = repository — shows whether an engineer concentrates on one repo or spreads across many.

Where the data lives:

- **Table:** `reviews`, grouped by author and repository together — one row per person per repo they touched.
- Authored reviews only; trailing 90 days.
- **Watch the repository count.** An org with dozens of repos produces an unreadable stack of colours. Keep the top few per engineer and roll the rest into an "other" bucket rather than emitting every one.

Vega-Lite spec:

```
{
  "width": 700, "height": 340,
  "mark": {"type": "bar"},
  "encoding": {
    "x": {"field": "engineer", "type": "nominal", "sort": "-y"},
    "y": {"field": "reviews", "type": "quantitative", "stack": true},
    "color": {"field": "repository", "type": "nominal"}
  }
}
```

§10.3 Specific rule — "How much does LiveReview save versus alternatives?" (query #20)

- Shape: waterfall — stacked `bar` with a computed invisible base (`y`) and visible delta (`y2`) per step, `color` = positive/negative.

Where the data lives:

- **Table:** `loc_usage_ledger` — it carries both the review volume and the actual LLM cost per settled row. Two totals for the window is all the database needs to give you.
- **Everything else on this chart is an assumption, not data.** The "alternative cost" you are comparing against comes from a stated assumption (engineer-hours saved × a rate), and the subscription cost is a known constant. Neither is in the schema.
- **Because of that, state your assumptions in the description.** A savings figure whose inputs are invisible is not persuasive to the person who has to defend it — quote the rate and the hours you assumed so they can argue with the number rather than distrust it.
- The invisible base segment that positions each waterfall bar is arithmetic on those totals, done before the chart.

Vega-Lite spec:

```
{
  "width": 600, "height": 380,
  "mark": {"type": "bar", "size": 60},
  "encoding": {
    "x": {"field": "label", "type": "nominal", "sort": null, "axis": {"labelAngle": -20}},
    "y": {"field": "base", "type": "quantitative"},
    "y2": {"field": "top"},
    "color": {"field": "color", "type": "nominal", "legend": null,
      "scale": {"domain": ["positive", "negative"], "range": ["#39d353", "#ff5c7c"]}}
  }
}
```

§10.4 Specific rule — "How much code has LR reviewed?" (long-span, compact view) (query #21)

- Shape: horizon graph — 2-3 `area` layers of the SAME field, each clamped to a band via SQL/app logic, stacked with rising opacity so intensity reads at a glance without a tall, sparse line. Reach for this only when the span is long (30-90+ points) and vertical space is scarce — §1's plain line+rolling-average is the default for shorter spans.

Where the data lives:

- **Table:** `loc_usage_ledger` alone — no join needed, since the org filter and the LOC figure both live on it.
- Daily sums, settled rows only, zero-filled across the full date series exactly as in §1.1.
- **The band-splitting is arithmetic on top of the daily value,** done after the query: pick a band height, then derive one column per band holding that band's share of the day's value. Three columns, three area layers. The query itself stays a plain daily series.

Vega-Lite spec (3 area layers, same field, rising opacity, band-clamped y-domain):

```
{
  "width": 800, "height": 90,
  "layer": [
    {"mark": {"type": "area", "color": "#7c9cff", "opacity": 0.35, "interpolate": "monotone"},
     "encoding": {"x": {"field": "day", "type": "temporal"}, "y": {"field": "b1", "type": "quantitative", "scale":
    {"mark": {"type": "area", "color": "#7c9cff", "opacity": 0.6, "interpolate": "monotone"},
     "encoding": {"x": {"field": "day", "type": "temporal"}, "y": {"field": "b2", "type": "quantitative", "scale":
    {"mark": {"type": "area", "color": "#7c9cff", "opacity": 1.0, "interpolate": "monotone"},
     "encoding": {"x": {"field": "day", "type": "temporal"}, "y": {"field": "b3", "type": "quantitative", "scale":
  ],
  "resolve": {"scale": {"y": "shared"}}
}
```

§10.5 Specific rule — "Are people trusting the reviews?" (query #26)

- Shape: diverging bar — up-votes positive, down-votes negative, sharing one categorical (engineer) axis and a zero line. Grounded directly in `review_feedback`'s `vote_type` column.

Where the data lives:

- Table:** `review_feedback` joined to `reviews` for the author name.
- Start from the feedback table, not from reviews** — every row you want is a vote, and reviews with no feedback have nothing to contribute to this chart.
- Exclude retracted votes**, and filter on the feedback's own timestamp rather than the review's; a vote cast this week on an old review is this week's signal.
- Group by author and vote type**, returning both as positive counts. The negation that puts down-votes below the zero line is a presentation step applied afterwards, not something the query should bake in.
- Feedback is sparse in most orgs. If the totals are tiny, say the actual counts in the description — "4 up, 1 down" is honest where a chart alone implies a trend that five data points cannot support.

Vega-Lite spec:

```
{
  "width": 600, "height": 320,
  "mark": {"type": "bar"},
  "encoding": {
    "y": {"field": "engineer", "type": "nominal"},
    "x": {"field": "n", "type": "quantitative"},
    "color": {"field": "vote_type", "type": "nominal",
      "scale": {"domain": ["up", "down"], "range": ["#39d353", "#ff5c7c"]}}
  }
}
```

§10.6 Exception — "What kinds of engineering problems is LiveReview finding?" (treemap requested, bar delivered) (query #17)

- Vega-Lite has no native treemap or Sankey mark. Per the standing rule in `internal/mcpagent/prompts/analytics_plan.md`: do not attempt to fake one — answer with a sorted `bar` (count per category, optionally colored by severity) instead. That is a faithful, honest answer to the same question, just not the requested picture.
- Also blocked on the same `ai_comments.content` JSON-payload issue noted in §9.1 — category/severity extraction needs to be reliable first.

§10.7 Exception — "What does a healthy engineering-review workflow look like?" (connected scatterplot, not built) (query #28)

- Would be: a `line` layer (points ordered by period via `"order"`, not SQL row order) plus a `point` layer at the same x/y, tracing successive weekly states so the trajectory itself becomes meaningful. Vega-Lite supports this pattern explicitly; it just hasn't been built against real data yet.